REPLIT AGENT BUILD BRIEF

Smart Crisis Intelligence System — Hackathon Prototype

---

1. APP OVERVIEW

App Name: CrisisIQ — Smart Hotel Crisis Response System

One-line description: An AI-powered, accessibility-first crisis detection and coordination platform for hotels that detects emergencies, dispatches staff, guides guests to safety, and adapts automatically to every guest's disability needs.

Core problem it solves: In a hotel emergency, staff have no intelligent coordination tool — they rely on radio calls and manual judgment. Guests with disabilities are completely overlooked. CrisisIQ automates detection, dispatch, multilingual alerts, and accessibility-aware evacuation guidance so every second counts and no guest is left behind.

Target users:

- Hotel managers (use the coordination dashboard)
- Hotel staff (receive dispatch notifications via staff app view)
- Hotel guests (trigger SOS, receive alerts, get guided to safety)
- Hackathon judges (see a live, impressive, polished demo)

---

2. FULL FEATURE LIST

Phase 1 — Core Foundation

Feature 1: Accessibility Profile at Check-In When a guest checks in, they are shown an optional accessibility form with five disability flags: Deaf/Hard of Hearing, Wheelchair User, Visually Impaired, Non-Verbal, and Cognitive Disability. This profile is saved to Firebase Firestore under their guest record. Every downstream feature reads this profile and adapts automatically — no manual override needed during an emergency.

Feature 2: Manual SOS + AI Emergency Type Classifier Every guest and staff member sees a large red SOS button on their screen. When pressed, the system calls the Gemini API with the

input context (voice transcript, typed description, or selected category) and classifies the emergency as Fire, Medical, or Security. It auto-populates the incident form with emergency type, room number, floor, timestamp, and the guest's accessibility flags.

Feature 3: Auto Staff Dispatch by GPS Proximity Staff locations are tracked in real time via browser geolocation, stored in Firebase. When an SOS fires, the system calculates the nearest staff member using Haversine distance formula, assigns them a role (First Responder, Escort, Coordinator), and sends them an in-app alert with exact task details — room, emergency type, and guest needs — all in under one second.

Feature 4: Live Coordination Dashboard A full-screen manager dashboard shows a real-time floor plan map of the hotel. Color-coded pins mark active incidents (red), staff positions (blue), resolved zones (green), and guests flagged for assistance (purple with disability icon). All updates stream live via Firebase Realtime Database. Staff accept/complete tasks from here. Incident status changes are timestamped.

Feature 5: Smart Evacuation Routing Using a visual floor plan grid overlaid on the dashboard, the system shows the safest escape routes from each room. Dangerous zones are marked and routes dynamically reroute around them. Wheelchair mode (auto-triggered from guest profile) removes all staircase routes, showing only ramp and accessible exit paths highlighted in a distinct color.

Phase 2 — AI Intelligence Layer

Feature 6: Multilingual Alert System Guest check-in profile stores preferred language. When an alert fires, the system uses the Google Translate API (or Gemini's multilingual capability) to instantly translate evacuation instructions into the guest's native language. Alerts are displayed on-screen and sent as push notifications simultaneously in 30+ languages, each guest receiving their own version.

Feature 7: AI Calm Coach — Voice Agent for Panicking Guests When a guest presses SOS, a simulated Gemini-powered voice agent interface activates on their screen. A chat/voice UI shows the AI speaking calmly — typing out instructions in real time: "Stay calm. I am alerting staff now. Stay low if there is smoke. Move to your door and feel for heat before opening." The agent asks clarifying questions to determine the guest's exact location and relays answers to the dashboard.

Feature 8: Emotion and Crowd Panic Detection (Simulated) A dedicated "Security Camera Feed" panel on the dashboard simulates a lobby camera. Using a pre-recorded or canvas-animated crowd scene, a MediaPipe-style visual overlay runs a real-time anomaly score meter. When the simulated score crosses a threshold (triggered by a demo button), a silent pre-alert fires to security staff — demonstrating the concept powerfully for judges without requiring a live camera.

Feature 9: Post-Incident AI Debrief Report After an incident is marked resolved, a "Generate Report" button calls the Gemini API. Gemini receives the full incident log (timestamps, staff actions, dispatch times, guest locations) and generates a structured debrief: executive summary, timeline, response time analysis, what worked, what failed, and three specific improvement recommendations. The report is displayed on screen and can be downloaded as a PDF.

Phase 3 — Accessibility Features

Feature 10: Deaf Guest — Silent Multi-Channel Alert When an emergency fires and a guest is flagged as Deaf, their alert screen switches to a full-screen high-contrast flashing banner (red/white alternating) with large bold text instructions. The phone vibration API fires a coded haptic pattern. A visual indicator on the dashboard shows the guest has been reached via visual alert channel, not audio.

Feature 11: Wheelchair Guest — Accessible Routing and Dedicated Escort When a wheelchair-flagged guest is in an active incident zone, their evacuation map removes all staircase paths and highlights only ramp/accessible exit routes in purple. The nearest staff member is auto-assigned as a dedicated escort — their notification reads "ESCORT REQUIRED — wheelchair guest, Room 412." The dashboard shows a wheelchair icon pin on their room.

Feature 12: Visually Impaired Guest — Spatial Audio Guidance When a visually impaired guest is affected, the Calm Coach voice agent (Feature 7) switches to an enhanced spatial audio script mode. The on-screen interface shows the AI delivering precise directional instructions: "You are in Room 308. Take 5 steps forward. Turn left 90 degrees. Walk 12 steps. The stairwell door handle is on your right." A simulated "phone call" UI frame makes this feel real for the demo.

Feature 13: Non-Verbal Guest — One-Tap Pictogram SOS Guests flagged as non-verbal see a different home screen: four large pictogram buttons replacing the standard SOS button. Each shows a universally recognized icon and single word — FIRE (flame icon), MEDICAL (cross icon), HELP (hand icon), UNSAFE (shield icon). One tap sends a complete SOS with their room, emergency type, and non-verbal flag — no typing, no speaking, no explanation required.

Bonus Features (Unique / Judge-Impressors)

Feature 14: Social Media Crisis Radar Panel A live panel on the manager dashboard simulates monitoring nearby social media for crisis keywords. Pre-loaded mock tweets appear in real time (scrolling feed) with geo-tags near the hotel — "smoke smell near Grand Hotel #emergency" — and Gemini summarizes the threat level as Low / Medium / High. A "Pre-Alert Security" button appears when threat crosses Medium, demonstrating external threat awareness.

Feature 15: Guest Wellness Check (Post-Incident) 30 minutes after an incident is resolved (or triggered by a "Send Wellness Check" button in the demo), the system auto-generates a personalized wellness message for affected guests using Gemini — acknowledging the event, offering support resources, and asking if they need assistance. The message is shown in the guest's language. A log of all sent messages with guest acknowledgement status appears on the dashboard.

---

3. TECH STACK

Frontend

- **React.js** (Create React App or Vite) — component-based UI, fast to build
- **Tailwind CSS** — utility-first styling, polished look with zero custom CSS effort
- **Framer Motion** — smooth animations for alerts, transitions, and panel reveals
- **React Router v6** — multi-page navigation

Backend / Real-time

- **Firebase Firestore** — guest profiles, incident logs, staff records
- **Firebase Realtime Database** — live dashboard updates, staff GPS pings
- **Firebase Authentication** — simple email/password login for Manager and Staff roles
- **Firebase Cloud Messaging (FCM)** — push notifications to staff devices

AI / APIs

- **Google Gemini API (gemini-1.5-flash)** — emergency classification, calm coach, debrief report, wellness messages, social radar summarization
- **Google Translate API** — multilingual alert translation
- **Google Maps JavaScript API** — floor plan overlay, evacuation route visualization
- **MediaPipe (browser WASM)** — crowd anomaly demo panel (pose detection simulation)
- **Web Speech API** — browser-native voice input for SOS voice trigger
- **Web Vibration API** — haptic coded alerts for deaf guest simulation

Utilities

- **jsPDF** — PDF generation for debrief reports
- **date-fns** — timestamp formatting throughout the app
- **Lucide React** — icon library for all UI icons
- **React Hot Toast** — non-intrusive notification toasts for alerts

Hosting

- **Replit** — development and live demo environment
- All API keys stored in Replit Secrets (environment variables)

---

4. PAGES & USER FLOW

Page 1: Landing / Role Selection (`/`)

Purpose: Entry point. User selects their role to enter the correct experience. **Content:**

- CrisisIQ logo and tagline: "Every second counts. Every guest matters."
- Three large cards: Manager Dashboard, Staff View, Guest Portal
- Clicking any card navigates to the login screen for that role **Navigation:** → Login Page

---

Page 2: Login (`/login`)

Purpose: Simple authentication for Manager and Staff roles. Guests skip login. **Content:**

- Email + password form
- Role is pre-set based on which card was clicked on Landing
- Demo accounts pre-filled: manager@hotel.com / staff@hotel.com
- "Continue as Guest" button links directly to Check-In page **Navigation:** → Manager Dashboard or Staff View or Check-In

---

Page 3: Guest Check-In (`/checkin`)

Purpose: Guest registers their room and accessibility needs. **Content:**

- Form fields: Name, Room Number, Preferred Language (dropdown of 20 languages)
- Accessibility section with five large toggle cards (tap to select):
    - Deaf / Hard of Hearing
    - Wheelchair User
    - Visually Impaired
    - Non-Verbal
    - Cognitive Disability

- "Check In" button saves to Firestore and navigates to Guest Portal **Navigation:** → Guest Portal

---

Page 4: Guest Portal (`/guest`)

Purpose: Guest's main screen during their stay. Adapts based on accessibility profile. **Content (Standard Guest):**

- Large red SOS button (center of screen)
- Current room number and floor displayed
- Recent hotel alerts panel (scrolling)
- Language selector (updates all text)

Content (Non-Verbal Guest):

- Standard SOS button is replaced by four large pictogram buttons (FIRE, MEDICAL, HELP, UNSAFE) in a 2x2 grid

On SOS press:

- Emergency type selector appears (or auto-sent for non-verbal)
- Gemini classifies and confirms the type
- Calm Coach panel slides up from the bottom (chat-style AI voice agent)
- Alert fires to Manager Dashboard and nearest staff

Navigation: → Calm Coach panel (inline), Evacuation Guide panel (inline)

Page 5: Calm Coach Panel (Inline Overlay on Guest Portal)

Purpose: AI voice agent that guides the panicking guest. **Content:**

- Chat-style interface with AI "speaking" (typing animation)
- Simulated call UI frame (phone call aesthetic)
- For Visually Impaired guests: enhanced spatial instruction mode with step-by-step directions
- "I'm Safe" button resolves the session
- All conversation relayed to dashboard in real time

Page 6: Manager Dashboard (`/dashboard`)

Purpose: Full operational view of the entire hotel during and between incidents. **Content:**

Top bar:

- Hotel name, current time, active incident counter
- Notification bell with live alert count

Left panel — Hotel Floor Map:

- Visual grid representing hotel floors
- Color-coded room pins: red (active SOS), blue (staff), purple (accessibility-flagged), green (resolved)
- Click any room to open its incident card
- "Block Zone" button marks areas as dangerous, triggers route recalculation

Center panel — Active Incidents Feed:

- Card per incident: guest name, room, emergency type, accessibility flags, time elapsed
- Staff assigned shown with status (Dispatched / En Route / Resolved)
- "Generate Debrief Report" button appears when incident is resolved

Right panel — Tabs:

- Tab 1: Staff Tracker — live list of all staff with GPS status and current assignment
- Tab 2: Social Radar — scrolling mock social feed with Gemini threat summary
- Tab 3: Guest Roster — all checked-in guests with accessibility flags visible
- Tab 4: Wellness Checks — post-incident message log

Bottom bar:

- "Simulate Emergency" button (for demo purposes) — triggers a pre-configured incident
- "Camera Feed" panel toggle — opens the crowd anomaly detection demo

Navigation: All interactions are inline on this page. Debrief Report opens in a modal.

Page 7: Staff View (`/staff`)

Purpose: Staff member's mobile-friendly task view. **Content:**

- Current assignment card (large, full-width): room, task, guest name, accessibility flag
- "Accept Task" / "Mark Complete" action buttons
- My Location toggle (activates GPS tracking)
- Incoming alert banner (slides in from top when new task assigned)
- History of completed tasks today **Navigation:** Standalone page, receives real-time updates from Firebase

---

Page 8: Debrief Report Modal

Purpose: Post-incident AI-generated report. **Content:**

- Triggered from Manager Dashboard after incident resolved
- Full-screen modal with:
    - Executive summary paragraph
    - Timeline table (action, actor, timestamp)
    - Response time metrics (cards: avg dispatch time, resolution time, guests assisted)
    - What worked / what failed (two columns)
    - Three AI recommendations (numbered list)
- Download as PDF button (jsPDF)
- Close button returns to dashboard

---

5. UI & DESIGN INSTRUCTIONS

Color Scheme

- **Primary background:** #0F172A (deep navy — conveys authority and calm)
- **Surface cards:** #1E293B (slightly lighter navy)
- **Primary accent:** #EF4444 (emergency red — for SOS, alerts, danger zones)
- **Success / Safe:** #22C55E (green — resolved zones, confirmed safe)
- **Warning:** #F59E0B (amber — pre-alerts, medium threat)
- **Accessibility accent:** #A855F7 (purple — all disability-related UI elements)
- **Info / Staff:** #3B82F6 (blue — staff pins, information panels)
- **Text primary:** #F8FAFC (near white)
- **Text secondary:** #94A3B8 (muted slate)

Typography

- **Font:** Inter (Google Fonts) — clean, professional, highly readable

- **Headings:** Inter 700, tracked slightly wide
- **Body:** Inter 400
- **Monospace data (timestamps, IDs):** JetBrains Mono

Layout Style

- Dark theme throughout — conveys seriousness, reduces eye strain during crisis
- Dashboard uses a three-column grid layout (left 25% / center 45% / right 30%)
- Guest portal is single-column, centered, mobile-first
- All cards have subtle border: 1px solid rgba(255,255,255,0.08)
- Rounded corners: 12px on cards, 8px on buttons
- No gradients except on the SOS button (radial red glow pulse animation)

Animations & UI Moments

- SOS button: constant slow pulse animation (scale 1.0 → 1.05 → 1.0, 2s loop)
- Alert firing: screen-edge red flash (0.3s) + toast notification slides in from top right
- Dashboard incident card: slides in from right with Framer Motion spring animation
- Calm Coach messages: typewriter effect (character by character reveal)
- Staff dispatch notification: full-width banner slides down from top of Staff View
- Floor map pins: bounce animation when new SOS fires
- Threat meter (Social Radar): animated progress bar that fills from left to right
- Pictogram SOS buttons (non-verbal): gentle scale-up on hover, satisfying press animation

Specific UI Components

- SOS Button: 180px diameter circle, bright red, white lightning bolt icon, "SOS EMERGENCY" label below
- Pictogram buttons: 150x150px cards with large SVG icon (top 60%) and single word (bottom 40%)
- Accessibility profile toggles: icon + label + colored border when selected (purple)
- Incident card: left-side colored bar (red/amber/green by status), guest name bold, flags as colored pill badges
- Floor map: CSS grid or SVG-based, rooms as rectangles, corridors as gaps, animated dot markers
- Calm coach: iOS-style chat bubbles, AI messages on left in dark card, user messages on right in blue

---

6. DATA & APIS

Firebase Data Models

Collection: `guests`

```
{
  id: string,
  name: string,
  roomNumber: string,
  floor: number,
  language: string,          // e.g. "ar", "ja", "es", "en"
  checkInTime: timestamp,
  accessibility: {
    deaf: boolean,
    wheelchair: boolean,
    visuallyImpaired: boolean,
    nonVerbal: boolean,
    cognitive: boolean
  }
}
```

Collection: `incidents`

```
{
  id: string,
```

```
  guestId: string,
  roomNumber: string,
  floor: number,
  type: "fire" | "medical" | "security",
  status: "active" | "in_progress" | "resolved",
  triggeredAt: timestamp,
  resolvedAt: timestamp | null,
  assignedStaffId: string,
  accessibilityFlags: string[],
  calmCoachLog: [
    { role: "ai" | "guest", message: string, timestamp: timestamp }
  ],
  timeline: [
    { action: string, actor: string, timestamp: timestamp }
  ]
}
```

Collection: `staff`

```
{
  id: string,
  name: string,
  role: "security" | "medical" | "coordinator",
```

```
  location: { lat: number, lng: number },

  status: "available" | "dispatched" | "busy",

  currentIncidentId: string | null,

  lastPing: timestamp

}
```

Realtime DB: `/alerts/{incidentId}`

```
{

  type: string,

  roomNumber: string,

  floor: number,

  guestName: string,

  accessibilityFlags: string[],

  firedAt: number   // Unix timestamp

}
```

Mock / Pre-Populated Data

Pre-load the following guests into Firestore on app start if collection is empty:

```
const mockGuests = [

  { name: "Ahmed Al-Rashid", roomNumber: "312", floor: 3, language: "ar",

    accessibility: { deaf: true, wheelchair: true, visuallyImpaired: false, nonVerbal: false, cognitive:
false } },

  { name: "Sofia Lindqvist", roomNumber: "308", floor: 3, language: "sv",

    accessibility: { deaf: false, wheelchair: false, visuallyImpaired: true, nonVerbal: false,
cognitive: false } },
```

```
  { name: "Raj Mehta", roomNumber: "315", floor: 3, language: "hi",

    accessibility: { deaf: false, wheelchair: false, visuallyImpaired: false, nonVerbal: false,
cognitive: true } },

  { name: "Chen Wei", roomNumber: "210", floor: 2, language: "zh",

    accessibility: { deaf: false, wheelchair: false, visuallyImpaired: false, nonVerbal: true,
cognitive: false } },

  { name: "Maria Garcia", roomNumber: "405", floor: 4, language: "es",

    accessibility: { deaf: false, wheelchair: true, visuallyImpaired: false, nonVerbal: false,
cognitive: false } },

  { name: "James Thompson", roomNumber: "101", floor: 1, language: "en",

    accessibility: { deaf: false, wheelchair: false, visuallyImpaired: false, nonVerbal: false,
cognitive: false } }

];

const mockStaff = [

  { name: "Priya S.", role: "medical", location: { lat: 1.001, lng: 1.001 }, status: "available" },

  { name: "David K.", role: "security", location: { lat: 1.003, lng: 1.002 }, status: "available" },

  { name: "Aisha M.", role: "coordinator", location: { lat: 1.002, lng: 1.004 }, status: "available" }

];
```

Mock social radar feed (pre-loaded array, displayed with timed reveal):

```
const mockSocialFeed = [

  { user: "@travelblog_sg", text: "Weird smoke smell near Grand Plaza hotel area... anyone
else? #singapore", time: "2 min ago", threat: "medium" },

  { user: "@local_news_feed", text: "Reports of commotion outside hotel district. Stay alert.
#cityalert", time: "5 min ago", threat: "low" },
```

{ user: "@hoteltraveler", text: "Fire truck just passed Grand Plaza!! Hope everyone's ok 🚨", time: "8 min ago", threat: "high" }

];

APIs

Google Gemini API

- Endpoint: `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent`
- Used for: Emergency classification, calm coach responses, debrief report generation, wellness messages, social radar threat summarization
- Auth: API key in request header / query param
- Store key in Replit Secrets as `VITE_GEMINI_API_KEY`

Google Translate API

- Endpoint: `https://translation.googleapis.com/language/translate/v2`
- Used for: Translating emergency alerts into guest's native language
- Auth: API key query param
- Store key in Replit Secrets as `VITE_TRANSLATE_API_KEY`
- Free tier: 500,000 characters/month

Firebase

- Initialize with project config stored in Replit Secrets:
  - `VITE_FIREBASE_API_KEY`
  - `VITE_FIREBASE_AUTH_DOMAIN`
  - `VITE_FIREBASE_PROJECT_ID`
  - `VITE_FIREBASE_RTDB_URL`
  - `VITE_FIREBASE_APP_ID`

Web Speech API (browser-native, no key needed)

- Used for: Voice SOS input on Guest Portal
- `window.SpeechRecognition` or `window.webkitSpeechRecognition`

Web Vibration API (browser-native, no key needed)

- Used for: Coded haptic alerts on deaf guest screens
- `navigator.vibrate([300, 100, 300, 100, 300])` — fire pattern

jsPDF (npm package)

- Used for: Generating downloadable debrief report PDF
- Install: `npm install jspdf`

---

7. STEP-BY-STEP BUILD INSTRUCTIONS FOR REPLIT AGENT

Follow these steps exactly in order. Do not skip any step. Complete each step fully before moving to the next.

STEP 1: PROJECT SETUP

Create a new React project using Vite:

npm create vite@latest crisisiq -- --template react

cd crisisiq

Install all dependencies in one command:

npm install firebase react-router-dom framer-motion @radix-ui/react-dialog lucide-react react-hot-toast date-fns jspdf tailwindcss postcss autoprefixer

Initialize Tailwind:

npx tailwindcss init -p

Configure `tailwind.config.js` to scan all React files. Set `darkMode: 'class'`. Add Inter and JetBrains Mono to `index.html` via Google Fonts link tags.

Set up `tailwind.config.js` with the custom color palette:

- emergency: #EF4444
- safe: #22C55E
- warning: #F59E0B
- accessibility: #A855F7
- staff: #3B82F6
- surface: #1E293B

- base: #0F172A

Create a `.env` file and add all Replit Secret keys using `import.meta.env.VITE_*` syntax throughout the app.

STEP 2: FIREBASE CONFIGURATION

Create `src/firebase.js`. Initialize Firebase app with environment variables. Export:

- `db` — Firestore instance
- `rtdb` — Realtime Database instance
- `auth` — Firebase Auth instance

Create `src/seedData.js`. Write a function `seedFirestoreIfEmpty()` that checks if the `guests` and `staff` collections exist. If empty, populate them with all mock data defined in Section 6. Call this function once inside `App.jsx` on mount using `useEffect`.

STEP 3: ROUTING STRUCTURE

In `src/App.jsx`, set up React Router with these routes:

- `/` → `LandingPage`
- `/login` → `LoginPage`
- `/checkin` → `CheckInPage`
- `/guest` → `GuestPortal`
- `/dashboard` → `ManagerDashboard`
- `/staff` → `StaffView`

Wrap the entire app in a `<Toaster />` from react-hot-toast for global toast notifications.

Apply global dark background `bg-[#0F172A]` and `text-[#F8FAFC]` to the root `div` in `App.jsx`.

STEP 4: LANDING PAGE

Build `src/pages/LandingPage.jsx`.

Display:

- Centered logo: shield icon (Lucide) + "CrisisIQ" in large bold white text
- Tagline: "Every second counts. Every guest matters." in muted slate text
- Three role cards in a horizontal row (stack vertically on mobile):
  - Manager Dashboard — building icon, description "Full crisis coordination view"
  - Staff View — user-check icon, description "Receive and manage emergency tasks"
  - Guest Portal — heart-pulse icon, description "Check in and access emergency help"
- Each card: dark surface background, colored left border (blue / green / purple), hover lifts slightly with Framer Motion
- Clicking Manager or Staff → `/login?role=manager` or `/login?role=staff`
- Clicking Guest → `/checkin`

STEP 5: LOGIN PAGE

Build `src/pages/LoginPage.jsx`.

Read the `role` query parameter from the URL. Display a centered card with:

- "Manager Login" or "Staff Login" as heading
- Email input pre-filled with demo credential
- Password input pre-filled with "demo1234"
- "Sign In" button — calls `signInWithEmailAndPassword` from Firebase Auth
- Below the form: small text "Demo credentials are pre-filled"

On successful login:

- If role is manager → navigate to `/dashboard`
- If role is staff → navigate to `/staff`

Create both demo accounts in Firebase Auth beforehand using a `createDemoAccounts()` utility function called on app mount.

STEP 6: GUEST CHECK-IN PAGE

Build `src/pages/CheckInPage.jsx`.

Display a centered, card-style form:

- Heading: "Welcome — Tell us how to keep you safe"
- Input: Full Name
- Input: Room Number
- Dropdown: Preferred Language (20 options: English, Arabic, Spanish, French, Mandarin, Hindi, Japanese, German, Portuguese, Russian, Italian, Korean, Turkish, Dutch, Swedish, Polish, Thai, Vietnamese, Bengali, Malay)
- Section heading: "Accessibility Needs (Optional)"
- Five toggle cards in a 2-column grid (tap to select, purple border when active):
    - Deaf / Hard of Hearing (ear icon)
    - Wheelchair User (wheelchair icon)
    - Visually Impaired (eye-off icon)
    - Non-Verbal (message-x icon)
    - Cognitive Disability (brain icon)
- "Check In" button: saves guest doc to Firestore, saves guest data to localStorage for session persistence, navigates to `/guest`

STEP 7: GUEST PORTAL

Build `src/pages/GuestPortal.jsx`.

On mount, load guest data from localStorage.

Standard guest layout:

- Top bar: room number, floor, language indicator
- Center: massive red SOS button (180px circle) with pulse animation
    - CSS animation: `@keyframes pulse { 0%,100% { box-shadow: 0 0 0 0 rgba(239,68,68,0.4) } 50% { box-shadow: 0 0 0 20px rgba(239,68,68,0) } }`
- Below SOS: "Hotel Alerts" panel — subscribe to Firebase RTDB `/alerts` node and show last 3 alerts as cards

Non-verbal guest layout (if `accessibility.nonVerbal === true`):

- Replace SOS button with 2x2 grid of four large pictogram buttons:
    - FIRE (flame icon, red background)
    - MEDICAL (cross icon, green background)
    - HELP (hand icon, blue background)
    - UNSAFE (shield-alert icon, amber background)

- Each button is 150x150px with icon (48px) on top, label below

On SOS press (or pictogram press):

1. Show emergency type selector modal (if not non-verbal) with three buttons: Fire, Medical, Security
2. Write incident to Firestore `incidents` collection
3. Write alert to Firebase RTDB `/alerts/{incidentId}`
4. Call Gemini API to classify and confirm type
5. Slide up the `CalmCoachPanel` component from the bottom

Calm Coach Panel:

- Slides up using Framer Motion (`y: 100% → y: 0`)
- Chat-style interface, dark card background
- AI messages appear one character at a time (typewriter effect using `setInterval`)
- Initial message auto-displays: "I'm here with you. I've already alerted the hotel team. Can you tell me your exact location — are you in your room right now?"
- Input field at bottom for guest to type responses
- Each guest message triggers a Gemini API call with conversation history as context, system prompt: "You are a calm emergency voice assistant in a hotel. Guide the guest to safety. Ask for their location. Give short, clear instructions. Be reassuring. Never panic."
- "I'm Safe" button at top right closes the panel and marks session complete

For Visually Impaired guests:

- Calm Coach opens automatically without SOS press (triggered when an active alert is detected in their floor/zone)
- First message uses spatial script mode: includes step counts and directional instructions based on their room

For Deaf guests:

- On any incoming alert: trigger `navigator.vibrate([300,100,300,100,300])` and display full-screen flashing overlay (alternating red and white `backgroundColor` CSS transition at 500ms intervals) with large white text of the alert message

---

STEP 8: MANAGER DASHBOARD

Build `src/pages/ManagerDashboard.jsx`.

This is the most complex page. Use a three-column CSS grid layout:

- Left: 25% width
- Center: 45% width
- Right: 30% width

Top Navigation Bar:

- Left: Shield icon + "CrisisIQ" logo + "Manager Dashboard" label
- Center: Live clock (updates every second with `setInterval`)
- Right: Active incident count badge (red, pulls from Firestore) + notification bell + logout button

Left Panel — Hotel Floor Map:

Build a visual floor plan using CSS grid or SVG. Represent a 5-floor hotel. Each floor is a horizontal row with 6 room rectangles (rooms 01–06 per floor). Room rectangles are 50x40px with room number text inside.

Color rooms based on live Firestore data:

- Room with active incident → red background, pulsing
- Room with accessibility-flagged guest → purple left border
- Room with assigned staff nearby → blue dot overlay
- Resolved room → green background (fades after 5 seconds)

On click of any room: open a side drawer showing that room's guest profile and current incident (if any).

Below the map:

- "Block Zone" button: click, then click a room to mark it as dangerous (room turns dark red with X icon, routes around it)
- "Simulate Emergency" button (amber, for demo): triggers a pre-configured incident for Room 312 (Ahmed — Deaf + Wheelchair) with type "Fire"

Center Panel — Active Incidents Feed:

Subscribe to Firestore `incidents` where status != "resolved".

For each incident, show a card:

- Left colored bar: red (fire), green (medical), blue (security)

- Guest name (bold, large)
- Room number and floor
- Emergency type badge
- Accessibility flags as colored pill badges (purple — use relevant icon per flag)
- Time elapsed since trigger (live counter, updates every second)
- Assigned staff name and status indicator dot
- Two buttons: "View Details" (expands timeline) / "Mark Resolved"

On "Mark Resolved": update Firestore status to "resolved", show "Generate Debrief" button.

Right Panel — Tabbed:

Build four tabs using Radix UI or plain state toggle:

Tab 1 — Staff Tracker:

- List all staff from Firestore
- Each row: avatar circle with initials, name, role badge, status dot (green=available, amber=dispatched, red=busy), current task if assigned

Tab 2 — Social Radar:

- Heading: "External Threat Monitoring"
- Threat level indicator: animated pill showing "LOW / MEDIUM / HIGH" with color
- Scrolling feed of mock social posts (reveal one every 8 seconds using `setInterval`)
- Each post: username, content, time, threat level colored dot
- "Gemini Threat Summary" box below: button "Summarize Threat" calls Gemini API with all mock posts and displays a 2-sentence threat assessment

Tab 3 — Guest Roster:

- Table of all checked-in guests from Firestore
- Columns: Name, Room, Floor, Language flag emoji, Accessibility flags (icon pills)
- Sort by floor number by default

Tab 4 — Wellness Checks:

- "Send Wellness Check to All Affected Guests" button
- On click: for each guest involved in resolved incidents, call Gemini API to generate a personalized message in their language, display the result in a card per guest
- Each card shows: guest name, room, generated message, "Sent" green badge

Camera Feed Toggle (bottom left):

- Toggle button "View Lobby Camera"
- Opens a panel showing a grey rectangle labeled "Lobby Feed — Live"
- Animated figure silhouettes (CSS) moving across the frame
- Anomaly Score meter: animated progress bar 0–100%
- "Simulate Panic Event" button: rapidly fills the bar to 87%, triggers a red pre-alert banner at the top of the dashboard: "⚠️ CROWD ANOMALY DETECTED — Pre-alerting Security"

STEP 9: STAFF VIEW

Build `src/pages/StaffView.jsx`.

Mobile-friendly, single-column layout.

On mount: load logged-in staff from Firebase Auth, get their staff doc from Firestore.

Display:

- Top bar: "CrisisIQ Staff" + staff name + role badge
- "My Location" toggle: when enabled, starts `navigator.geolocation.watchPosition()` and writes lat/lng to Firestore staff doc every 5 seconds
- Current Assignment Card (large, full-width, red border if active):
  - "No active assignment" (gray, when idle)
  - OR: room number (very large), guest name, emergency type, accessibility flags, "ESCORT REQUIRED" banner if wheelchair flag
  - "Accept Task" button (green) → updates assignment status to "accepted" in Firestore
  - "Mark Complete" button (blue) → updates incident status, clears assignment
- Alert banner: subscribe to Firebase RTDB `/alerts` — when new alert appears for this staff member's assignment, show an animated top banner: slides down, pulses red border, plays alert sound using Web Audio API (simple 880Hz beep)
- Task History: last 5 completed tasks from Firestore

STEP 10: DEBRIEF REPORT MODAL

Build `src/components/DebriefModal.jsx`.

Triggered when "Generate Debrief" is clicked on a resolved incident.

Display a full-screen modal overlay (dark background, white/surface card centered):

Loading state: spinner + "Gemini is generating your incident report..."

On mount, call Gemini API with this prompt:

"You are a hotel safety analyst. Generate a structured post-incident debrief report based on this incident data: [JSON of incident]. Include: 1. Executive Summary (2 sentences), 2. Timeline of events (as a JSON array with action/actor/time fields), 3. Response metrics (average dispatch time, resolution time), 4. What worked well (3 bullet points), 5. What failed or was slow (3 bullet points), 6. Three specific improvement recommendations. Return as JSON only."

Parse the JSON response and render:

- Executive summary paragraph
- Timeline as a styled table (action | actor | time columns)
- Metric cards in a 3-column grid: Avg Dispatch / Resolution Time / Guests Assisted
- Two columns: "What Worked" (green check icons) / "What Failed" (red X icons)
- Numbered recommendation cards (amber left border)

"Download PDF" button:

- Use jsPDF to generate a PDF with all report sections
- Include CrisisIQ header, incident ID, hotel name, date
- Download as `incident-report-{roomNumber}-{date}.pdf`

"Close" button dismisses modal.

STEP 11: MULTILINGUAL ALERT SYSTEM

Create `src/utils/translateAlert.js`.

Write an async function `translateAlertMessage(message, targetLanguage)`:

1. If targetLanguage is "en", return message unchanged
2. Call Google Translate API:

   POST https://translation.googleapis.com/language/translate/v2?key={API_KEY}

Body: { q: message, target: targetLanguage, format: "text" }

3. Return translated text

In the SOS trigger flow (Step 7), after writing to Firestore:

- Get guest's language from their profile
- Call `translateAlertMessage()` with the emergency instruction text
- Display translated text in the Calm Coach first message
- Write translated alert to Firebase RTDB so the dashboard shows the guest's language

---

STEP 12: GEMINI INTEGRATION UTILITY

Create `src/utils/gemini.js`.

Write a reusable async function `callGemini(prompt, systemInstruction = "")`:

```
const response = await fetch(

`https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${import.meta.env.VITE_GEMINI_API_KEY}`,

  {

    method: "POST",

    headers: { "Content-Type": "application/json" },

    body: JSON.stringify({

      system_instruction: { parts: [{ text: systemInstruction }] },

      contents: [{ parts: [{ text: prompt }] }]

    })

  }

);
```

const data = await response.json();

return data.candidates?.[0]?.content?.parts?.[0]?.text || "";

Import and use `callGemini()` in:

- GuestPortal (emergency classifier, calm coach)
- ManagerDashboard (social radar summarizer, wellness check messages)
- DebriefModal (report generation)

STEP 13: ACCESSIBILITY ROUTING LOGIC

Create `src/utils/evacuationRoutes.js`.

Define the hotel as a floor-room grid. Each room has an array of possible exit paths (sequences of rooms/corridors to traverse).

Write `getEvacuationRoute(roomNumber, floor, blockedZones, wheelchairMode)`:

- Returns an array of steps: `["Exit room 312", "Turn left", "Walk 15m to stairwell B", "Descend 3 floors", "Exit via main lobby"]`
- If `wheelchairMode` is true: replace all "stairwell" references with "elevator" or "ramp", and filter paths that go through stairs-only zones
- If a blockedZone is in the path: find alternate route that avoids that room/corridor

Use this function in:

- GuestPortal: show evacuation steps as a numbered list when SOS fires
- ManagerDashboard: show route overlay on floor map when a room is clicked during an active incident

---

STEP 14: FINAL POLISH & DEMO MODE

Add a floating "DEMO" pill button (bottom-right corner, amber) visible on the Manager Dashboard.

Clicking it opens a Demo Control Panel (side drawer):

- "Trigger Fire — Room 312 (Deaf + Wheelchair guest)" — fires a full incident simulation
- "Trigger Medical — Room 308 (Visually Impaired guest)" — fires medical incident
- "Show Crowd Anomaly Detection" — activates camera panel and auto-fills anomaly meter
- "Send Wellness Check" — triggers wellness check flow for all guests
- "Generate Sample Report" — opens debrief modal with sample data

This panel allows judges to see every major feature within 3 minutes without manual navigation.

STEP 15: RESPONSIVE & FINAL CHECKS

Ensure these work correctly:

1. Guest Portal is fully usable on a 375px mobile screen (stack everything vertically)
2. Manager Dashboard is usable on a 1280px+ desktop screen
3. Staff View is fully usable on a 375px mobile screen
4. All Gemini API calls have loading states (spinner) and error states (toast notification)
5. All Firebase reads have loading skeletons (gray pulsing rectangles)
6. The app does not crash if API keys are missing — show a friendly "API key not configured" message
7. The "Simulate Emergency" button on the dashboard is always visible and works without any prior setup
8. Toasts appear for every major action: SOS fired, staff dispatched, alert translated, report generated

---

DEMO FLOW FOR JUDGES (3 minutes)

1. Open Manager Dashboard → point out live floor map and guest roster (accessibility flags visible)
2. Click "Simulate Emergency — Room 312" → watch incident card appear, staff auto-dispatched, floor map pin turn red
3. Open Guest Portal as Ahmed (Deaf + Wheelchair) → show silent strobe alert instead of audio, coded vibration
4. Show Evacuation Route — staircase removed, ramp-only path highlighted in purple
5. Switch to non-verbal guest — show pictogram SOS buttons, tap FIRE → incident fires in 1 tap
6. Open Calm Coach — show AI typing step-by-step instructions in real time
7. Show Social Radar tab — mock tweets appearing, click "Summarize Threat" → Gemini responds

8. Mark incident as Resolved → click "Generate Debrief" → Gemini builds report, download PDF
9. Close with: "Every guest protected. Every second accounted for. This is CrisisIQ."

End of Brief — paste this entire document into Replit Agent to begin building.